

基于信息缝与权限划分的 可验证多智能体 RL 任务构造

一种把智能体任务库改造成多智能体 RL 任务的方法，并在 AppWorld 上完整实现

王锦敖

Multi-Agent Task Forge

2026 年 7 月 17 日

摘要 本文给出的是一套方法，以及一个被完整跑通的实例。该方法把原本并非为多智能体设计的智能体任务库，改造成必须通过协作才能完成的训练任务，同时完全保留原任务的环境与判定器，不新写任务、不新写环境、更不新写奖励。它对任务库只提出四项要求：判定器由他人编写、环境可安装可执行、每条任务附带参考解、动作面可以沿角色边界切开。任何满足这四项的任务库都可以套用本方法；AppWorld 是本文实际运行的那一个，而 §6.6 会明确指出：它也是唯一的一个。具体到 AppWorld：我们从中读取 147 条带完整参考信息（ground truth）的任务，分析参考解中的跨应用数据流，并用静态污点分析筛出 39 条确实存在信息依赖的任务。每道题需要哪些角色，完全由参考解实际调用的应用决定。改造后，主智能体不再持有应用 API，只能把工作委派给各应用的专属智能体（specialist）；参考信息和判定器则保存在主智能体无法访问的辅助容器（sidecar）中。最终奖励仍由 AppWorld 原有的数据库状态检查给出，不使用大模型判定器。

这条链路中，批量生成的开销并不高：选题、角色划分和 Harbor 目录渲染都由静态分析完成，39 条任务可以直接生成 78 个配置实例。真正的成本发生在生成之后。每条任务都需要用参考解证明其可解，每个配置也需要在多个随机种子下重复运行，才能判断它是否提供稳定、非退化的训练信号。目前只有 3 条参考路径。因此，下一步不应继续扩充约束配置，而应先自动生成参考路径（§6.3），再扩展源任务环境。AppWorld 当前只能提供 13 个相对独立的题族；39 条任务和 78 个目录并不代表同等规模的任务多样性。

目录

1 问题与目标	1
2 能力选择与学习顺序	1
3 从原始任务到协作任务	2
3.1 从真实依赖中选择题目	3
3.2 把单智能体任务改成协作任务	3
4 造题方法：四个关键环节	3
4.1 不变量：优先继承已有判定器	3
4.2 算子：如何产生候选任务	4
4.3 判据：什么样的候选可以保留	5
4.4 良率：从候选到可用数据	5
5 运行环境、轨迹与验证	7
5.1 容器边界	7
5.2 一条任务如何运行	7
5.3 奖励与有效性参照	8
6 难度控制与规模化	9
6.1 难度的控制维度	9
6.2 低成本环节：静态分析与批量渲染	9
6.3 主要成本之一：参考解	9
6.4 主要成本之二：多随机种子验证	9
6.5 AppWorld 的规模上限：13 个独立题族	10
6.6 准入条件：本方法对任务库的要求	10
7 数据样本与实验结果	11
8 讨论与后续延展	13
8.1 当前结果与下一步验证	13
8.2 从空间约束延展到时间约束	13
9 结语	14

1 问题与目标

一条强化学习任务通常由指令、可交互环境和奖励三部分组成。对单智能体任务来说，验证往往很直接：文件是否生成、数据库是否更新、测试是否通过，都可以由程序检查。多智能体任务多了一层要求。除了完成最终目标，主智能体还要找到合适的分工，并把必要的信息准确地传给不同角色。

角色数量并不能自动带来协作。如果主智能体仍能直接调用全部工具，或任意一个子智能体都能独立完成任务，那么多出来的角色只是界面变化，并没有改变问题本身。另一种做法是直接评价对话是否“像在合作”，但这种奖励高度依赖主观判断，也很难稳定地扩展到大规模数据。

我们的选择是从已有可靠判定器的真实任务出发，只改变工具权限和信息分布，使协作成为完成任务的必要条件。这样既能迫使模型规划、委派和传递信息，又能继续用最终环境状态计算奖励。

把这个目标提到它真正应当被复用的层次上，它就不是一批数据，而是一套**方法**。目前并不存在“多智能体轨迹 + 环境 + 免费判定器”的现成语料，因此凡是要造这类数据的人，都必须自己制造多智能体结构；真正的选择只有一个：制造哪一层。制造判定器，错误就会隐形——因为你自己的测试会认同你自己的错误，§4.1 正是这一失败被测出来的记录。只制造约束——它改变谁能做什么，从不改变什么算做完——失败模式就退化成“题目太易或太难”：可见，且可以用 §4.3 的判据定价。本文余下的部分，就是这套方法的四个动作——准入任务库 (§6.6)、挖掘信息缝 (§3.1)、施加约束 (§3.2)、只接受落在下界与对照之间的实例 (§4.3)——以及被这四个动作完整跑通的一个任务库：AppWorld。

2 能力选择与学习顺序

多智能体协作涉及多种能力，但并不是每一种都适合直接作为 RL 维度。为了得到清晰的训练信号，我们优先选择能够由环境结果验证、并且可以逐层组合的能力。MAST 将常见失败归纳为系统设计、智能体间失配、任务验证和终止等类别 [4]；MultiAgentBench 则比较了星形、链式、树形和图结构等通信拓扑下的协作表现 [3]。这些因素彼此相关，造题时需要分阶段引入，而不是一次全部叠加。

表 1：当前优先训练的能力

能力	模型需要学会什么	如何观察失败
任务拆解与依赖识别	把一个目标拆成若干可执行子任务，并判断哪些中间结果必须先获得	遗漏必要应用、顺序错误、下游缺少上游信息
角色分配	根据子智能体拥有的信息和工具，把子任务交给合适的角色	请求发给错误角色，或要求一个角色访问它看不到的信息
跨角色通信	在委派中补充实体、时间范围和上一步结果，使局部执行服务于全局目标	子智能体完成了自己的请求，但最终世界状态仍然不正确
并行调度	在满足依赖和资源约束的同时减少不必要的串行等待	资源冲突、漏排任务、依赖未满足或总完成时间过长

当前交付直接覆盖表 1 中的前三项。并行调度虽然也可以通过环境状态验证，却不适合放进这套 AppWorld 环境：辅助容器采用单线程执行，委派调用是串行阻塞的，而 AppWorld 本身也不是线程安全的。现有奖励 $r = N_{\text{pass}}/N$ 只描述终态，无法评价总完成时间。要训练并行调度，需要另一个真正支持并发的运行时，因此不在本次交付范围内。

课程可以按依赖关系逐步展开。第一层只要求模型识别角色并完成一次正确委派；第二层加入跨应用依赖，要求主智能体把上游结果带到下游；第三层隐藏角色的能力说明，让主智能体自行判断“谁可能知道什么”；之后再加入委派预算、更多角色和更复杂的通信拓扑。第三层实际上已经涉及一部分心智理论 (Theory of Mind)：主智能体必须估计其他角色掌握的信息，而不能默认对方与自己共享上下文。不过，当前奖励只能判断最终任务是否完成，无法确认一次失败究竟来自心智建模、任务拆解还是消息表达，因此这里不把心智理论和动态恢复单独列成训练维度。

更细的失败归因仍然可以作为诊断工具。例如，让教师模型阅读交互记录，判断问题出在拆解、路由还是沟通。但这相当于新增了一个判定器，必须先与人工标注对照，报告一致率，才能知道它有多可信。在完成这项验证之前，终态分数仍应是唯一的训练奖励，教师模型最多用于分析失败和设计后续课程。本文尚未进行这项研究，因此表中只保留能够由终态直接验证的能力。

建议的进阶顺序为：单次角色路由 → 星形拓扑、能力文档可见 → 星形拓扑、仅角色名称可见 → 有限预算与多级依赖 → 链式/树形拓扑与运行时故障。

当前实现覆盖到第三层。委派预算、多级依赖和复杂拓扑已有设计方向，但在进入训练集之前，还需要分别验证可解性和稳定的难度增量。

3 从原始任务到协作任务

核心思路可以概括为：保留任务和判定器，只重构智能体访问任务的方式。

原始任务已经给出了用户指令、交互环境和程序化判定器。改造时，我们只在环境外层加入权限、可见性、通信拓扑和委派预算等约束。模型可以采用不同的拆解和通信策略，但最终仍由原任务的测试判断是否完成。

更具体地，记原始任务为

$$\tau = (x, \mathcal{E}, J), \quad \mathcal{E}_c = W(\mathcal{E}; c), \quad \tau_c = (x, \mathcal{E}_c, J).$$

其中 x 表示用户指令， $\mathcal{E}$ 表示原始环境， J 是程序化判定器。包装算子 W 只改变环境，不改写指令和判定器，因此等式两边的 x 与 J 保持不变。约束

$$c = (P, V, G, B)$$

的四个分量是： P 权限划分（哪个角色持有哪个应用的 API）， V 角色可见性（主智能体能够看到哪些角色信息）， G 通信图（谁可以向谁发消息）， B 委派预算。

这里的 P 描述划分方式：主智能体不持有任何应用 API，每个专属智能体只绑定一个应用。具体需要哪些应用，不由配置预先指定，而是从参考解中推导 (§3.1)。权限划分是其余约束成立的前提；如果主智能体仍能直接访问全部工具，可见性、拓扑和预算都不会真正限制它。判定器 J 读取辅助容器中的终态，而 W 保证主智能体所在的文件系统中没有 J 或完整参考信息 (§5.1)。

这一思路与 SWE-smith 复用仓库既有测试的原则相近 [2]：优先继承可靠的验证逻辑，而不是为每道新题重新定义“什么算完成”。整个过程分为三步：识别任务中的跨应用依赖，据此确定角色集合，再把同一任务渲染成不同的协作配置。

3.1 从真实依赖中选择题目

AppWorld 包含 9 个日常应用、457 个 API 和 750 条复杂任务（当前发布的数据集中实际可见 732 条），并通过数据库状态测试判断任务是否完成、是否引入了额外修改 [1]。本项目读取训练集和开发集中带完整参考信息的 147 条任务，并分析参考解实际调用的应用。

角色集合由任务本身决定。例如，参考解同时调用 phone 和 venmo，就为这两个应用各设置一个专属智能体。用于提交答案的 `supervisor.complete_task` 不提供业务信息，因此不计入角色。按这一规则，147 条任务中有 51 条涉及至少两个真实应用；其余任务直接过滤，不额外添加无关角色。

3.2 把单智能体任务改成协作任务

改造后的环境包含一个主智能体（Main）和若干应用专属智能体。主智能体能看到完整用户目标，却不能调用任何应用 API；phone 专属智能体只能访问 phone，venmo 专属智能体只能访问 venmo。每个专属智能体只接收主智能体发来的局部请求，并不知道完整任务。

例如，任务要求“在我的 Venmo 社交动态里，给今天所有涉及室友的交易点赞”。主智能体必须先向 phone 查询室友名单，再把姓名和日期范围交给 venmo。phone 掌握联系人关系，却不能操作 Venmo；venmo 可以点赞，却不知道哪些人是室友。两者都无法独立完成任务，信息必须沿下面的路径传递：

phone 获取室友姓名 → 主智能体传递姓名与范围 → venmo 遍历并执行操作。

这类任务同时要求模型判断信息所在的角色、安排有依赖的执行顺序，并在委派时带上足够的上下文。题面仍是一条自然语言指令，依赖结构只存在于环境和参考解中，不会直接展示给模型。需要注意的是，终态奖励只能判断整体结果，无法把一次失败单独归因到上述某项能力（见 §8.1）。

4 造题方法：四个关键环节

SWE-smith 最值得借鉴的并不是如何注入 bug，而是它把验证问题交还给仓库原有的测试：先破坏一个本来能运行的仓库，再用既有测试判断修复是否成立 [2]。沿着这条思路，一套可规模化的造题方法需要回答四个问题：什么保持不变、候选如何生成、什么样的候选可以保留，以及每种生成方式最终能留下多少有效任务。SWE-smith 对五类算子分别测得 56%、35%、40.2%、33.8% 和 96.9% 的良率。

下面按照这四个环节说明本项目的设计，也明确区分已经实现的部分和仍需验证的部分。

4.1 不变量：优先继承已有判定器

造题既可以重建任务世界和判定器，也可以只改变智能体受到的约束。两种做法的风险并不相同。约束设置不当，任务会变得过易或不可解，这通常能从对照实验中发现；判定器一旦写错，奖励就会稳定地衡量错误目标，而且自测未必能暴露问题。因此，本项目只改变“谁能访问什么”，不重新定义“什么算完成”。这样做的目的不是简化实现，而是尽量把失败留在可观测、可比较的部分。

这并不意味着语言模型不能出现在造题链路中。它可以生成候选任务，也可以充当环境中的专属智能体，或在没有程序化判定器时辅助诊断。区别在于：一旦模型参与打分，就需要先用人工标注测量其一致率。这个要求同样适用于任何自建规则，包括 §4.4 用来筛选任务的信息熵分析；

它目前已有代码和结果，但误差率还没有单独评估。

因此，已有的程序化判定器仍是首选。其他方案并非不可行，只是需要额外承担判定器验证的成本。

源环境选择。 源环境需要满足几个基本条件：有可复用的程序化判定器；环境真实、可安装、可执行；至少训练集提供完整参考信息，以便分析任务结构和验证可解性；工具空间还要能够沿两个以上的角色边界切分。

实践中还需要再加一条：角色边界必须同时形成信息边界。若两个角色可以各自完成工作，只需安排先后顺序，那么任务考查的是分派，而不是信息协作。只有当下游动作依赖上游角色独有的事实时，委派和通信才会影响最终结果。本文把这种跨角色的信息依赖称为“信息缝”。共享同一文件系统的软件仓库通常较难形成这种边界，因为信息对所有角色同时可见，角色划分容易退化为文件分工。

4.2 算子：如何产生候选任务

候选任务的生成方式分为程序化变换、语言模型改写和任务组合三类，见表 2。

表 2：候选生成方式与当前实现状态

产生方式	SWE-smith 的对应	本项目的算子	改题/改 配置	良率	状态
程序化	Procedural (13 种 AST 变换)	partition : 按参考解触及的应用划分角色，主智能体不持有任何 API	改配置	39/51 选题；3/3 参考解通过	已交付
程序化	Procedural	blind : 隐藏角色能力说明，只留角色名	改配置	0/3 有效 实例	已交付
程序化	Procedural	route (委派预算): 限制允许的委派次数	改配置	未测	未交付
程序化	Procedural	route (链式/树形拓扑): 改变角色之间的通信路径	改配置	0/3 有效 实例	未实现
程序化	Procedural	perturb : 运行中撤下一个角色，迫使重规划	改题	—	未实现
语言模型 改写	LM Modify / LM Rewrite	weaken : 抽走指令中的日期与实体，移入辅助容器的隐藏用户状态	改题	—	未实现
组合	Combine	combine : 把共享实体的两道题合成一道	改题，判定器需重 构	—	未实现

“未交付”表示功能已有实现，但尚未进入发布配置；“未实现”则表示代码链路还不完整。委派预算属于前者：`partition.py` 已在主智能体和辅助容器两侧执行预算约束，但当前发布配置均未设置上限。链式拓扑属于后者：`partition.py` 虽定义了专属智能体之间的交接关系，却没有实际调用点，因此现有 `chain` 任务不可解，相关分数不能解释为难度。表中的算子名称用于描述

方法，对应代码中的 `Topology`、`Visibility` 和 `delegation_budget` 等参数。

需要区分“新任务”和“新配置”。`partition` 与 `blind` 只改变同一道题的访问方式，因此 39 条任务乘以 2 个配置得到的是 78 个运行单元，而不是 78 道彼此独立的新题。SWE-smith 的 50k 条实例来自对任务本身的修改。若要在本项目中获得同样意义上的题量增长，就必须引入会改变任务内容的算子，并重新测量其良率。

`combine` 尤其需要谨慎。AppWorld 的判定器会检查副作用，甲任务中的“不得新增 Venmo 交易”可能被乙任务的合法操作触发，因此简单地取 $J_A \wedge J_B$ 并不能得到自洽的新判定器。组合任务可能带来更高产量，也最容易破坏“继承判定器”这一前提。

4.3 判据：什么样的候选可以保留

SWE-smith 的保留规则很直接：应用候选补丁后运行测试，只有原本通过的测试变红，候选才算有效。整个过程由程序完成，不需要额外判断。

本项目使用类似的夹逼规则。记无动作基线为 r_0 ，无约束对照为 r_{open} ，某个受约束配置的得分为 r 。只有满足

$$r_0 < r < r_{\text{open}}$$

并且 $r_{\text{open}} > r_0$ 时，这个配置才提供可解释的训练信号。若 $r \leq r_0$ ，它在奖励上无法与不采取行动区分，原因可能是任务过难，也可能是运行链路故障；若 $r \geq r_{\text{open}}$ ，说明约束没有带来可测影响，得到的仍是基础任务能力；若无约束对照本身落在基线上，则基础任务或运行环境已经有问题，不能再从受约束分数中解释难度。

可解性需要单独验证，不包含在上述夹逼规则中。参考解必须在容器内实际运行并通过判定器，结果再统一汇总到实验记录中。

这条规则已经写入造题链路，可以直接通过命令行执行。这样做很有必要：项目早期曾把 0.333 解释为权限划分带来的难度，后来才确认它只是无动作基线。

4.4 良率：从候选到可用数据

良率描述候选中最终有多少可以使用。本项目有两次筛选，因此需要分别看两个比例：一是题库中有多少任务具备真实的跨角色信息依赖，二是这些任务在具体配置下有多少能产生训练信号。

选题良率：51 条多应用任务中有 39 条信息缝任务

`select.py` 最初使用 `len(roster) >= 2` 作为筛选条件。它只能说明参考解调用了至少两个应用，却不能证明两个应用之间存在信息依赖。若第二个应用只是独立执行另一项操作，增加角色并不会真正考查通信。

`seams.py` 对参考解执行前向污点分析，检查一个应用产生的事实是否被另一个应用的动作使用：它可能直接出现在调用实参中，也可能出现在控制该动作的条件分支中。该过程完全基于静态分析，不需要模型调用，结果保存在 `sweep/appworld_seams.json`。

表 3: 选题良率: 三层过滤

过滤	保留	占比
宽松筛选: 把 <code>supervisor.complete_task</code> 也算作协作者	147/147	100%
严格筛选: 剔除提交通道 (<code>select.py</code> 当前的判据)	51/147	34.7%
+ 信息缝: 要求事实真的跨应用流动	39/147	26.5%

51 条多应用任务中有 12 条没有信息缝, 分别来自四个完整题族。即使其中两个题族与已交付样本同属 `phone+venmo` 组合, 它们也只需要把操作分给两个角色, 不需要传递跨应用事实。这样的任务可以用于训练分派, 却不能作为信息协作数据。

两轮过滤解决的是同一类问题。第一轮把保留率从 100% 降至 34.7%, 排除了只负责提交结果的 `supervisor`; 第二轮再降至 26.5%, 排除了不传递信息的第二个应用。若只做第一轮, 就会把 51 条候选中的 12 条, 也就是 23.5%, 误认为协作任务。

信息依赖不一定直接出现在函数实参中。分析结果显示, 有 15 条任务只通过条件分支传递依赖, 例如上游结果决定 `if ... : continue` 之后的动作是否执行; 已交付的三条任务都属于这一类。若只检查实参, 会漏掉整个交付题族。

配置良率: 渲染后的任务是否具有训练信号

将 §4.3 的夹逼规则应用到现有实验, 得到表 4。

表 4: 按夹逼规则检查现有结果 (每个配置实例一个随机种子)

配置	_1	_2	_3	有效实例	说明
star-docs	持平	持平	0.833	1/3	唯一产生梯度的实例
star-names	持平	基线下	基线下	0/3	两个实例因如实报告失败而低于基线
chain-names (未实现)	基线	基线	基线下	0/3	已知的假信号

两个已交付配置共形成 6 个配置实例, 目前只有 1 个落在有效区间。`star-docs` 的均值 0.944 来自一个有效实例和两个与对照持平的实例; `star-names` 的均值 0.445 则由一个持平实例和两个低于基线的实例组成 (原因见 §7)。这些运行均正常结束, 没有系统错误; 但从 RL 数据角度看, 落在基线以下或与对照持平的实例都不能提供预期的协作训练信号。

两轮实验的均值变化很大, 但逐个实例得到的结论没有变化。`star-names` 的均值从 0.778 降到 0.445, 相差 0.333; 然而每个实例的有效性分类保持不变。这说明总体均值容易掩盖不同实例的失败方式, 判断可训练性时应保留实例级结果。

最后, 这些数字仍然只来自单个随机种子, 不能当作良率估计。一次运行无法区分偶然失败与稳定落在基线附近的实例。要得到可信的良率, 需要增加每个配置实例的随机种子数量, 而不是继续增加配置种类。

5 运行环境、轨迹与验证

5.1 容器边界

Harbor 把每道题组织成独立任务，目录中包含指令、容器环境、测试和参考解 [5]。主智能体所在的容器只安装轻量的 `team` 客户端；AppWorld、专属智能体和任务状态全部运行在辅助容器中。主智能体因而只能通过委派改变环境，无法直接调用应用。

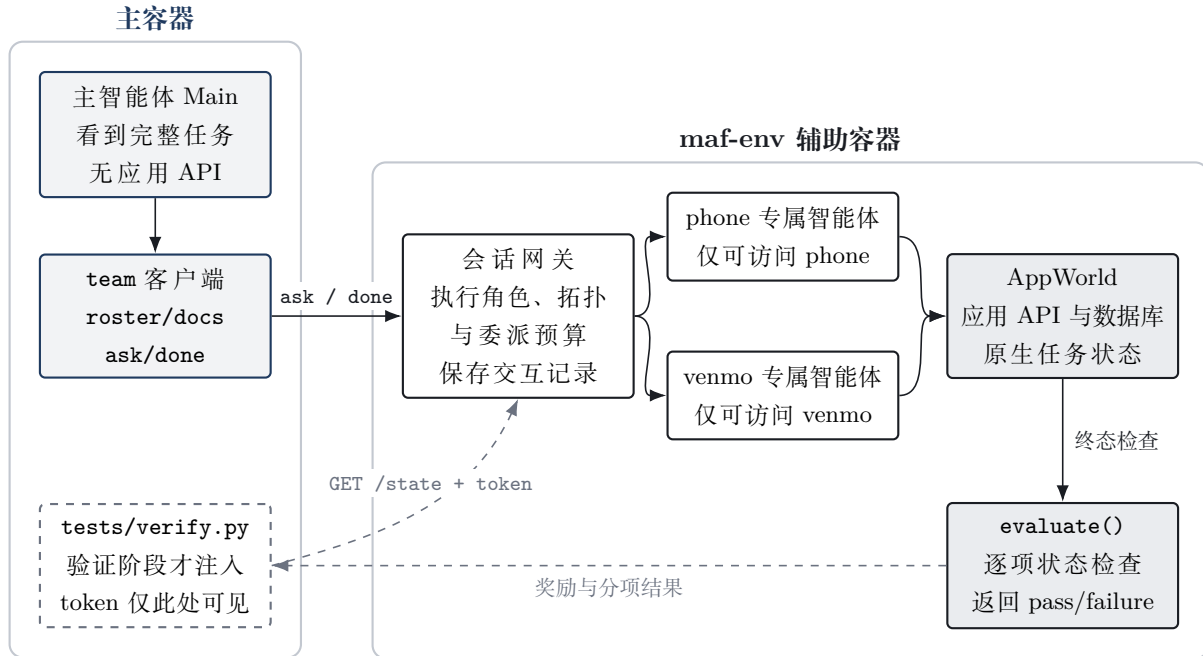


图 1: Harbor 任务的容器架构。左侧是主智能体可见的容器；AppWorld、专属智能体、交互记录和判定逻辑均位于辅助容器。虚线表示只在任务结束后启动的验证流程。

图 1 中有两条关键隔离边界。主智能体的文件系统不包含 AppWorld、完整参考信息或判定器，只能通过 `team` 发送自然语言请求。验证令牌在任务执行期间同样不可见；Harbor 只在任务结束后注入测试，由验证器读取辅助容器中的终态。权限限制由容器环境强制执行，主智能体无法在执行过程中读取自己的得分。

5.2 一条任务如何运行

图 2 给出了任务 `2a163ab_1` 的参考轨迹。一次 `team ask` 内部可能包含专属智能体的多轮推理和 API 调用，图中只展示角色之间传递的信息。任务能否完成，取决于主智能体是否先取得完整的室友名单，再把名单、日期范围和分页要求交给 `venmo`；消息数量本身并不重要。

用户任务 在我的 Venmo 社交动态里，给今天所有涉及室友的交易点赞。

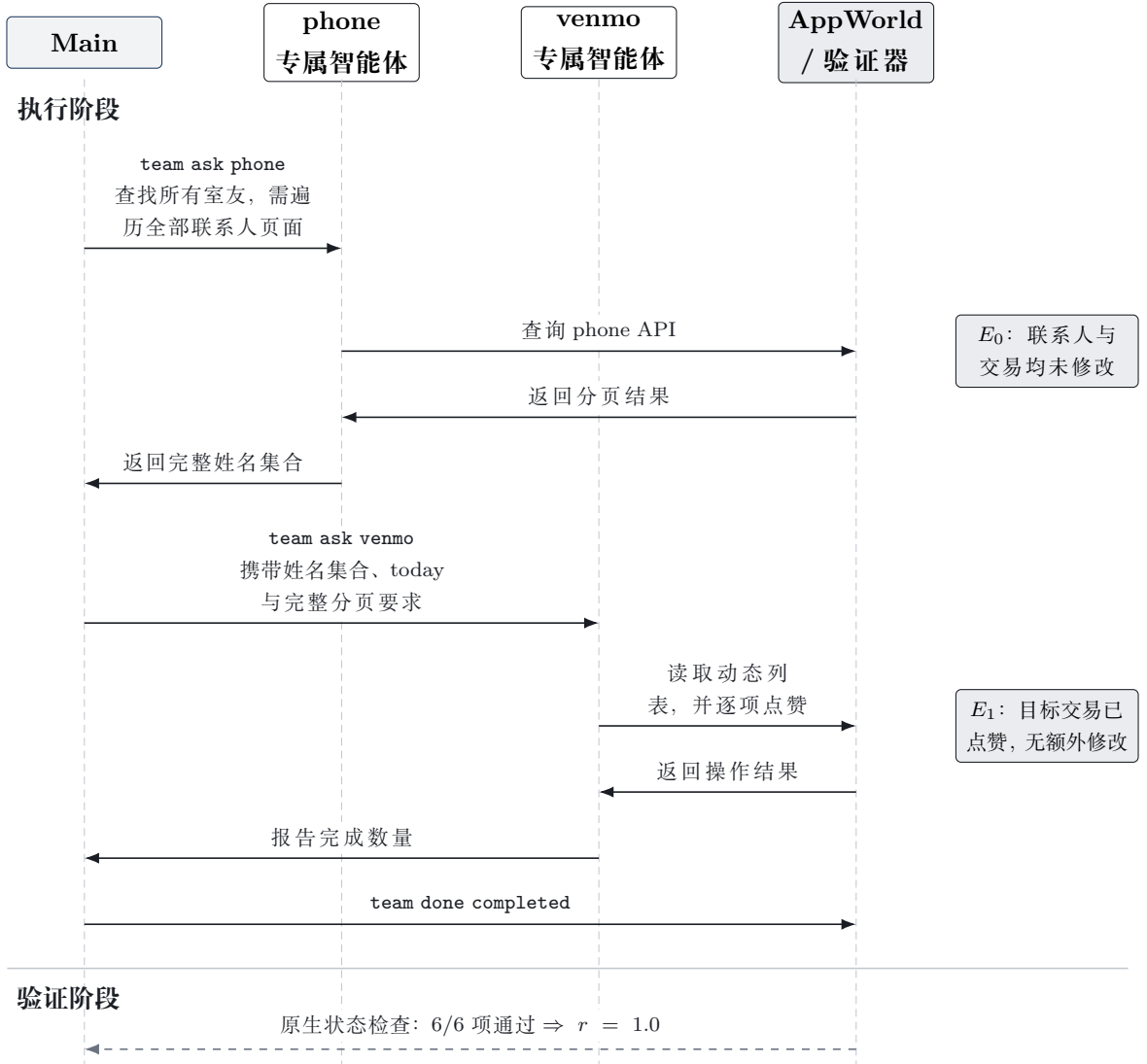


图 2: 参考轨迹中的信息流。上游角色取得事实，主智能体将其整理成下游可以执行的请求；奖励由终态 E_1 决定，不依赖轨迹文本。虚线表示任务结束后的验证阶段。

5.3 奖励与有效性参照

验证器不评价对话写得是否合理，也不依赖模型自行报告的工作状态。若任务共有 N 项状态要求，其中 N_{pass} 项通过，则奖励为

$$r = \frac{N_{\text{pass}}}{N}.$$

这个比例保留了部分完成度，同时避免引入大模型判定器。模型可以采用不同的拆解和通信路径；只要最终状态相同，且没有产生额外副作用，就会得到相同奖励。

每个配置都需要三个参照。参考解用于证明任务可解；无约束对照让单个智能体直接访问全部应用，用来确认基础任务本身没有问题；无动作基线则测量在不修改环境时能够自然通过多少检查。只有当受约束配置稳定地落在无动作基线与无约束对照之间，它才可能为 RL 提供有效梯度。

6 难度控制与规模化

6.1 难度的控制维度

难度可以从四个方向调整。其一是信息可见性，例如主智能体能否读取角色能力说明；其二是依赖结构，从两步顺序依赖扩展到三个应用或多分支依赖；其三是通信约束，包括委派预算以及星形、链式、树形等拓扑；其四是操作规模，例如分页数量、实体数量和状态修改数量。

这些因素应逐层加入。一个有效的难度调节项需要同时满足两点：参考解仍能稳定通过，模型表现相较上一层又出现可重复的变化。若某种拓扑只是使任务无法执行，它带来的是环境故障，而不是更高难度。当前链式配置正因不满足可解性而未进入交付 (§4.2)。

6.2 低成本环节：静态分析与批量渲染

造题流程的前半段由三条命令完成，均不调用模型。`measure` 从 147 条带完整参考信息的任务中解析参考解实际调用的应用；`seams` 进一步执行前向污点分析，保留 39 条存在跨应用事实流的任务；`render` 再将任务与约束配置写成 Harbor 目录。这些工作只涉及静态分析和文件生成，在普通笔记本上即可在数分钟内完成。

这一步成本较低，是因为 §3 中的 W 只改变环境，指令 x 和判定器 J 可以直接复用。39 条任务与 2 个已实现配置组合后，`render` 可以生成 78 个任务目录，并为每条任务保留一个无约束对照。

困难在于确认这些目录中有多少真正可解、可训。主要成本出现在渲染之后，分别来自参考解和多随机种子验证。

6.3 主要成本之一：参考解

Harbor 任务需要在 `solution/` 中提供参考解，以证明任务可解。这里的参考解并不是完整动作回放。专属智能体由真实语言模型驱动，主智能体只负责分解和传递信息，因此参考路径记录的是委派结构，例如先向 `phone` 查询姓名，再把结果交给 `venmo`；具体操作仍由专属智能体在线完成。这种参考解带有随机性，需要真实模型调用，也可能因某次执行失误而失败。

目前 `REFERENCE_PATHS` 只包含同一题族的 3 条手写路径。若直接渲染全部 39 条任务，会得到 78 个目录，其中 72 个 `solve.sh` 会明确提示缺少参考路径。没有通过参考解验证的任务不能进入训练集，因此，参考路径覆盖率是当前规模化的主要瓶颈。

这部分有明确的自动化路径。`seams.py` 已经能够识别事实的生产者、事实本身和事实的消费者；参考路径所需的结构与这个三元组基本一致，可以据此生成“向上游角色查询，再把结果交给下游角色”的委派骨架。剩余的人工知识主要是应用级调用约定，例如 `phone` 和 `venmo` 都需要显式要求遍历全部分页。这类规则按应用维护，而不是按任务逐条编写。任务 `2a163ab_3` 的一次失败也说明了这一点：`phone` 只读取第一页，漏掉了一名同事；在请求中补充分页要求后，问题便不再出现。

6.4 主要成本之二：多随机种子验证

配置是否有效取决于多次运行的结果分布，不能由单次结果决定 (§4.4)。按当前设定，每个配置实例至少需要 `SEEDS_FOR_YIELD = 5` 次重复。39 条任务、2 个配置和 5 个随机种子需要 390 次受约束运行；再加上 195 次无约束对照，总计 585 次。按现有实验记录的平均耗时估算，串行约需 20 小时；在没有速率限制的理想条件下，16 路并行约为 1.3 小时。

实际瓶颈主要有两个。第一，在线调用专属智能体会受到 API 速率限制；现有记录中，同类

配置实例的耗时可从约 200 秒增加到 6,700–15,000 秒，即 30–75 倍。第二，验证成本会随配置数量线性增长：每增加一个调节项，都需要在全部任务上重新进行多随机种子对照。因此，应该先在小规模样本上确认配置确实产生训练信号，再扩大数据量。

6.5 AppWorld 的规模上限：13 个独立题族

即使补齐参考解和多随机种子验证，AppWorld 本身的任务多样性仍然有限。39 条信息缝任务来自 **13 个题族，每族 3 个变体**。同一题族共享依赖结构，差别主要在实体、日期和操作规模。已交付的 2a163ab_1/2/3 就是同一题族的三个变体，分别涉及室友、兄弟姐妹和同事。按应用组合统计，集中度更高：24 条任务、8 个题族都属于 phone+venmo（表 5）。

表 5: 39 条信息缝任务的应用组合分布

应用组合	题数	题族数
phone + venmo	24	8
phone + simple_note + venmo	3	1
simple_note + spotify	3	1
file_system + phone + venmo	3	1
phone + splitwise + venmo	3	1
file_system + simple_note	3	1
合计	39	13

AppWorld 虽然包含 9 个应用，但这 147 条带完整参考信息的任务只涉及其中 6 个，39 条信息缝任务也只覆盖 6 种应用组合。因此，当前可训练的多样性更接近 13 个题族和 6 种组合，而不是 39 条任务或 78 个目录。配置数量、任务数量和题族数量需要分别统计；若要显著扩大数据规模，关键是增加源任务环境，而不是继续复制配置。

6.6 准入条件：本方法对任务库的要求

AppWorld 是本方法的输入，而不是它的前提。方法对任务库只提出四项要求，缺少任何一项，它都会退回到“自己设计判定器”，而这正是 §4.1 所论证的、本方法存在的理由所禁止的：

1. **不是你写的程序化判定器**。它使奖励能够批量计算，也避免重新定义“协作是否成功”。它的全部价值恰恰在于：已经有别人替你验证过它。
2. **真实、可执行、可安装**。而不是对某个世界的描述。一个无法 `pip install` 或 `docker pull` 的任务库，最后一定由你自己补完；补完之后它就是你设计的，第一项要求也随之作废。
3. **任务级参考解**。参考解既用于推导角色集合，也用于证明任务可解。AppWorld 的 750 条任务中只有 147 条提供完整参考信息，因此当前分析范围止于这 147 条。
4. **可切开的工具边界**。不同角色需要拥有彼此隔离的工具或数据；否则主智能体可以绕过协作，其他约束也失去意义。

准入是必要条件，而非充分条件。一个任务库可以四项全部满足，却依然产不出可用数据，因为这四项都没有度量信息缝有多深：判定器可以完美无缺，而每条任务仍然坍缩到单一角色上。这

正是信息缝分析 (§3.1) 必须是一次测量、而不是一个假设的原因；AppWorld 自己就差点说明了这一点——它 147 条带参考信息的任务里有 96 条是单应用题。信息缝也是“换环境”这件事不再免费的地方：SWE-smith 拥有这份名单上最强的判定器，其规模化也正依赖第一项要求 [2]，但它的缝是代码模块，因此在它上面做划分，得到的是一道多了几个智能体的软件工程题，而不是编排版。

关于普适性，把话说清楚。本方法中没有任何一步是为 AppWorld 定制的：准入条件是任务库的性质，角色集合读自参考解实际触碰的工具，约束作用于访问面，而有效性判据 (§4.3) 只需要一个分数、一个下界和一个对照。但这是一个从构造出发的论证——读代码即可核验——而不是一个结果。**本方法被端到端跑通的任务库，恰好只有一个。**因此这里主张的适用范围是准入条件的适用范围：任何满足全部四项要求的任务库都应当可行，而目前已知可行的是一个。换第二个环境是代价最低的证伪方式，而它尚未被执行；§8.1 把这件事当作待办，而不是当作免责声明。合理的扩展顺序是：先根据现有信息缝自动生成参考路径，在 13 个题族上完成良率验证，再迁移到更多源任务库，而不是继续在同一题族上增加配置。

7 数据样本与实验结果

当前交付选择了同一题族的三个实例。三道题都需要先从 phone 获取特定关系的联系人，再由 venmo 处理相应日期范围内的交易。它们共享同一依赖结构，但联系人关系、日期范围和操作规模不同。

表 6: 三条 Harbor 数据样本

Task ID	用户任务	必要协作
2a163ab_1	在 Venmo 社交动态里，给今天所有涉及室友的交易点赞	phone 找出室友姓名， venmo 遍历今天的动态并点赞
2a163ab_2	在 Venmo 社交动态里，给昨天所有涉及兄弟姐妹的交易点赞	phone 找出兄弟姐妹姓名， venmo 遍历昨天的动态并点赞
2a163ab_3	在 Venmo 社交动态里，给昨天或今天所有涉及同事的交易点赞	phone 找出同事姓名， venmo 跨两个日期遍历并点赞

三条样本的 Harbor 参考解均通过 6 项状态检查，奖励为 1.0。在此基础上，我们比较三种配置：open 是无约束对照；star-docs 强制主智能体委派，但允许读取角色能力；star-names 只显示角色名称，不提供能力说明。实验中，主智能体使用 gpt-5.6-sol，专属智能体使用 gpt-4.1；目前每个配置实例只运行了一个随机种子。

表 7: 约束消融结果

配置	平均奖励	三条任务	平均耗时/s	含义
open	1.000	1.000/1.000/1.000	30.2	一个智能体拥有全部应用
star-docs	0.944	1.000/1.000/0.833	200.8	强制委派, 可读角色的真实 API 目录
star-names	0.445	1.000/0.167/0.167	145.9	强制委派, 角色能力不可见
无动作基线	0.333	0.333/0.333/0.333	—	不修改环境, 但提交正确答案类型

表 7 的均值看似形成了清晰梯度, 但逐条检查后, 两段差异都不能直接解释为预期的难度变化。

star-docs 比 open 低 0.056, 说明强模型即使失去直接工具, 也能通过少量委派完成大部分任务。这里还修正了一个早期测量问题: 交付容器通过 `team docs` 提供真实 API 目录, 而旧的进程内实验实际只显示角色名称。补上长度分别为 2,838 和 5,444 字符的目录后, 平均分仍为 0.944; 委派次数却从 2–3 次增加到 3–5 次, 平均耗时也从约 100 秒升至 201 秒。就这三个样本而言, 能力文档改变了交互成本, 却没有带来可见的成功率提升。

star-names 从 0.944 降至 0.445, 也不能直接归因于能力可见性。三个配置实例中有两个得分为 0.167, 甚至低于 0.333 的无动作基线。原因是主智能体未找到目标交易后, 如实提交了“没有找到符合条件的交易”; AppWorld 的 `assert answers match` 对操作类任务期待的答案是 `None`, 因此这段说明额外丢失了一项检查。若提交 `completed`, 反而可以得到 0.333。

这个现象可以在不调用模型的情况下直接复现: 保持环境完全不变, 只修改最终提交文本, 再运行同一个判定器。项目中的离线探针已经完成了这项检查, 并把结果保存为可复现的实验记录。

表 8: 环境状态相同、仅提交文本不同时的得分

提交方式	得分	未通过的检查
什么都不做, 谎称 <code>completed</code>	0.333	4 项状态检查
什么都不做, 如实报告“没有找到”	0.167	同样那 4 项, 再加 <code>assert answers match</code>

两个策略都没有修改环境, 终态也完全相同; 差别只在最终提交文本, 却造成了 1/6 的分数差。

当前奖励会让虚假的完成声明比诚实的失败报告高 1/6。对训练中的策略而言, 这是一个方向相反的激励。

因此, 表 7 中的无动作基线 0.333 并不是严格下限, 而是“不修改环境、但提交正确答案类型”的得分。诚实报告失败仍可能低于它。

直接修改 AppWorld 判定器会破坏“继承原判定器”的前提, 因此当前交付没有改写这项逻辑。更稳妥的处理是先把协议项和环境状态项分别记录, 在训练阶段明确决定哪些分量可以进入策略奖励。现有标量 $r = N_{\text{pass}}/N$ 将两者混在一起, 所以当前所有难度结果都必须结合这一协议偏差解释。

因此，现有数据不足以证明“隐藏角色能力能够形成更好的难度梯度”。按照 §4.4 的夹逼规则，star-names 的有效实例为 0/3，star-docs 也只有 1/3。

8 讨论与后续延展

8.1 当前结果与下一步验证

目前的三个实例来自同一题族，每个配置也只运行了一个随机种子，因此 0.056 和 0.167 的差异都不能视为稳定效应。下一步应先为现有任务补充更多随机种子，估计分数差异和有效实例比例；随后再加入 phone-simple note、file system-spotify 以及三应用任务，检查结果能否跨题族复现。

还有第二个性质不同的缺口，而它比上面任何一项都更便宜：**本文把方法写成与任务库无关，却只在一个任务库上跑通过 (§6.6)**。信息缝分析恰好是其中最明显可迁移、代价也最低的一步——它不需要 rollout，也不需要任何模型调用，只需要参考解——因此，把它跑在第二个满足准入条件的任务库上并公布信息缝分布，要么佐证普适性主张，要么以一个下午的代价推翻它。在有人真正做完之前，这一主张依赖的是“读构造”而不是证据；本文选择如实标注，而不是让一张候选任务库的调研表，去干只有测量才能干的活。

终态奖励可以判断任务是否完成，却不能区分失败来自拆解、角色选择还是信息表达。因此，本阶段没有把 Theory of Mind 和动态恢复作为独立维度。更细粒度的归因可以借助成对 probe 或环境侧故障注入，但最终仍应落到可验证的状态差异，而不是只评价对话文本。

只有在基础星形配置经过多随机种子、多题族验证后，才适合继续加入委派预算和链式/树形拓扑。这样每一层课程都建立在上一层可解、可测的基础上，难度变化也更容易解释。

8.2 从空间约束延展到时间约束

当前工作主要改变协作的空间结构：工具属于谁、信息对谁可见、消息可以沿哪些路径传递。下一阶段可以加入时间维度，不再在任务开始时一次给出全部信息，而是在智能体主动询问或环境达到特定状态后再揭示。这样可以训练隐藏需求识别和用户干预下的重规划，同时继续复用现有终态验证。

最小实现可以从同一任务生成一组成对样本。对照组提供完整指令；实验组只给出符合真实使用习惯的不完整请求，把联系人范围、日期或兼容偏好保存在辅助容器的隐藏用户状态中。主智能体需要先识别信息缺口，再通过询问获得这些约束。两组任务共享初始环境和最终判定器，因此不必直接评价“问题问得好不好”：遗漏关键问题会导致终态错误，而无差别追问则会消耗有限的交互预算。

再进一步，可以在任务执行过程中加入用户干预。例如，智能体完成部分工作后，用户补充新的范围或限制，主智能体需要更新计划、复用仍然有效的结果，并把变化同步给相关角色。这里要区分两类信息：若后续补充只是初始需求中未说清的部分，原有判定器可以保持不变；若用户真正改变了目标，就必须为新终态提供同样可靠的环境验证，不能只根据对话判断是否适应成功。

这条时间轴扩展可以表示为

$$\tau_{c,h} = (x_0, \mathcal{E}_c, U_h, \Gamma_h, J),$$

其中 U_h 表示只存在于辅助容器的隐藏用户状态， Γ_h 描述信息被询问、揭示或修改的时机。这里仍需保持现有隔离边界：隐藏状态不能进入主智能体的文件系统，交互记录不能由智能体自行伪

造，最终奖励继续由可重放的环境状态决定。只有当完整指令、隐藏需求和中途干预三种配置形成稳定差异后，才适合把时间维度纳入正式课程。

9 结语

这项工作的核心，是从已有真实任务中识别跨角色依赖，再通过权限和信息划分让这些依赖成为完成任务的必要条件。主智能体负责规划、委派和传递上下文，专属智能体只执行局部工作；最终奖励仍由 AppWorld 的真实环境状态给出。

当前结果证明了两点：权限划分后的环境可以在 Harbor 中完整运行，三条参考解也都通过了 6 项状态检查。但它尚未证明这些约束能够形成稳定的难度梯度。现有实验只在一个 star-docs 实例上观察到非退化信号；star-names 没有形成可解释的难度增量，链式拓扑尚未实现，并行调度也不在本次交付范围内。

因此，下一步应先补足已交付配置的多随机种子结果，分离提交协议与环境状态的奖励影响，并把样本扩展到不同依赖结构。隐藏用户需求和中途干预是自然的后续方向，但它们应建立在一个已经证明能够稳定产生训练信号的基础配置之上。

参考文献

- [1] Harsh Trivedi et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. *ACL 2024*, arXiv:2407.18901. <https://arxiv.org/abs/2407.18901>
- [2] John Yang, Kilian Lieret, Carlos E. Jimenez, et al. SWE-smith: Scaling Data for Software Engineering Agents. arXiv:2504.21798, 2025. <https://arxiv.org/abs/2504.21798>
- [3] Kunlun Zhu, Hongyi Du, Zhaochen Hong, et al. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. *ACL 2025*, arXiv:2503.01935. <https://arxiv.org/abs/2503.01935>
- [4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025. <https://arxiv.org/abs/2503.13657>
- [5] Harbor Framework. Task Structure: Creating and Running Tasks for Agentic Environments. Documentation, accessed 2026-07-16. <https://www.harborframework.com/docs/tasks>